

What an Agile Architect Can Learn from a Hurricane Meteorologist

Eric Richardson, Chemical Abstracts Service

You'll have to read further to find out! There are so many interesting things about this article—it's the first in a series based on presentations from the SATURN conference. My first college degree was in chemistry, so there was an immediate connection for me with this story about chemical abstracts. The author brings us architectural wisdom, combined with an agile point of view and weather forecasting. Enjoy! —*Linda Rising, associate editor*

CHEMICAL ABSTRACTS SERVICE (CAS) receives data from 61 patent offices worldwide and more than 10,000 scientific journal titles. For approximately 150 years, the technology and processes associated with this data were paper-based—until two years ago, electronic publishing supplanted print in only a few cases. It has been astonishing to see an entire industry moving to electronic-only publication so

quickly, thereby steering architectural changes at a rapid pace as well.

One of the first segments to move to all-electronic publishing was the patent offices. The quantity of paper required to print patent documentation became insurmountable a long time ago, so patent offices started taking pictures of the documents and publishing the images. Similarly, the volume of CAS's journal subscriptions required us to start

taking pictures of pages, the result of which—virtual paper—represents an exact replica of the original “authority” publication for CAS scientists to analyze.

Today, however, we've also found a need to extract information from the picture. Many excellent optical character recognition (OCR) technologies are available in a hundred or so languages, but they fail when someone needs to extract a chemical structure, formula, or reaction. This is because there are no dictionaries that the character recognition program can use to check the accuracy of its work, meaning sometimes the OCR programs inadvertently introduce errors. The most frustrating thing about inaccurate OCR of chemical information is that at some point all the data were electronic before the publisher printed the magazine or the patent official made a picture of the patent application. We're left dealing with the accumulated error of transformation from data to image, then image to data. In this article, I discuss some of the challenges we face at CAS and how learning from a hurricane meteorologist helps you make better architectural decisions in a changing environment.

Setting the Stage

CAS receives electronic data feeds in different formats from different publishers, different formats from the same publisher, and lots of changes with or without notice. Scale this up from 61 patent offices to more than 10,000 journals, and you get a sense of the problem. To compound matters, with electronic publications, CAS might get several revisions instead of the “one true version” represented by

a final printed publication, post peer review and copyedit. A patent might come to us as an application, a revision, and, finally, a grant, and the content is usually different in each version. In the past, it was very easy to know when to put the content in front of a chemist for analysis. Today, we have to make a decision: Do we analyze it now, later, or both?

At CAS, we foresee a three- to five-fold increase in the number of article versions to be processed as we anticipate the somewhat longer-term elimination of physical, printed publica-

to this:

```
<author>
<name lang="EN">
<given>Eric</given>
<family>Richardson</family>
</name>
</author>
```

That kind of change requires someone to change the logic extracting the data. Sure, you could try to create a system that could handle any name representation you throw at it, but add addresses and scale to more than 100 languages,

Like meteorologists who forecast how a storm system develops and changes over time, agile architects forecast how software systems evolve.

tions. Our architecture has to handle the shift from the “one true version” that arrived in the mail to data feeds from publishers with items at different points in their life cycles. Our business processes will change from waiting for that final version to arrive, analyzing it once, and never looking at it again to incremental analysis performed several times as the article progresses through the publisher’s workflow.

When a publisher changes how it represents data, we might know about it in advance or when it first arrives in the new form. Scanned images are representation agnostic, meaning that an image doesn’t care if you name a segment “callout” or “sidebar,” or if the publisher switches from serif to sans serif or from Helvetica to Arial. Pixels are pixels. However, we very much care if the data feed goes from this:

```
<author>
<first-name>Eric</first-name>
<last-name>Richardson</last-name>
</author>
```

and the complexity quickly rivals natural language processing.

All software systems have an architecture, defined as “the structure or structures of the [program or computing] system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.”¹ Software architects attempt to lend predictability to the notoriously unpredictable realm of software design and development and take a longer, broader view than developers and project managers. They “are expected to facilitate software development projects as well as represent the overall system’s quality attributes.”²

CAS was an early information technology adopter, and consequently, we have a large base of legacy software. Our legacy systems are immensely valuable, but they’re based on paper—virtual paper now, but still paper workflows. They also carry some heavy baggage from the days of batch processing.

When we were first trying to incorporate electronic data feeds at CAS,

it became clear our virtual paper-oriented processes and systems weren’t up to the task. We tried to replace our content management system with one more appropriate, only to find our workflow was tightly coupled to the old system. In light of this discovery, we adjusted our path to enable incremental replacement of legacy systems, beginning with efforts to minimize the impacts of all paper workflows. The next step in our architecture’s evolution will put service facades on legacy systems, allowing us to decouple the consumer from the implementation. We don’t know exactly what will drive our business processes as we evolve, so we’re evaluating several alternatives. Fortunately, we have a good idea of what those processes are, which is where we’re taking immediate action. When we decide on a workflow engine, it can exploit the work we’ve already finished.

An Interesting Analogy

Like meteorologists who forecast how a storm system develops and changes over time, agile architects forecast how software systems evolve. Meteorologists use many different kinds of data sources as inputs to forecasting, and architects need to gather diverse data types and employ different techniques in their efforts. But unlike the meteorologist, who has well-established meteorological practices for forecasting, the agile architect has far fewer standard practices to follow.

Agile software development is about solving immediate needs, continually adapting and refactoring as problems arise. One of the four core values in the agile manifesto is “responding to change over following a plan.” Some agile practitioners regard architecture as overhead, an antique from the age of waterfall development methodologies. Yet, agile development has a critical need for architecture. If software developers didn’t need to produce working software on short schedules, they

wouldn't need agile development. If they didn't need to continue producing working software as the system evolves, they wouldn't need software architecture. Software architecture naturally gravitates to plans, designs, and structure, preferably before the writing of any code. Agile development naturally gravitates to expediency and fluid structures. How do we embrace the benefits of both approaches to create "good" architectures in an agile world?

We can look to the work of meteorologists for inspiration. They've had to learn about the behavior of large complex systems for much longer than computer scientists. The principles hurricane meteorologists use to assist in predicting the behavior of large, complex weather systems are applicable to large, complex software systems as well.

Sifting through the Data

If you give a hurricane meteorologist a giant pile of data about a storm spinning in the middle of the Atlantic Ocean and ask her to determine exactly where it will come ashore, she can analyze the data, construct a detailed and accurate model of the atmospheric conditions and weather patterns, run some simulations, and come up with a forecast—two months after the storm hits. It'll probably be wrong, but not by much, a moot point for the people in the storm's path. The problem isn't with the forecast's accuracy but with the time needed to prepare it. On the other extreme, if given a satellite image and a few points of other data—and a few minutes—a hurricane meteorologist can prepare a forecast so uncertain it might as well not even exist. There is a point where forecast accuracy and timeliness overlap. The model favored by hurricane meteorologists is to do just enough data acquisition and analysis to be reasonably certain what the storm will do and then start telling people to get ready.

The same holds true for designing software architecture. An architect can spend a great deal of time designing what he thinks is a perfect system only to present it when it's irrelevant. Big design up front (BDUF) is ignored in agile development in favor of two alternatives: no design up front (NDUF), which leads to accidental software architecture, or enough design up front (EDUF). But what is "enough" design?

Making a Forecast

Before agile architects can make any plans, they first have to decide on the target architecture. This is a very difficult thing to do because it has many factors to consider, the most important being business goals. Most businesses have a defined planning horizon, typically five years.³ Technological and cultural factors are also major considerations. How will an organization's culture respond to both business demands and technological change? Developing the target architecture requires the ability to speculate knowledgeably with these factors in mind.

its current state and forecasts its developmental path according to business goals, technology evolution, and other factors. Even in "green field" development at a new company, the current architecture is defined—it might not be well defined, but it's there.

Most software systems, like storms, evolve over time, and the architect must maintain system stability during that evolution. She knows the architecture will change just as a storm's path changes. Skilled at predicting where the system is going, the agile architect needs to be able to predict without accidents. Likewise, the architect's implementers must have concrete predictions or they'll improvise (and allow "accidents" to happen).

The Cone of Uncertainty

The farther away a storm, the more uncertain the forecast, which is what meteorologists call the *cone of uncertainty*. Hurricane meteorologists forecast a storm's projected path, knowing that the storm won't follow this path exactly. Therefore, when publishing a

Architects, especially agile architects, need to learn this—that it's okay to be a "little" wrong.

Even though it's impossible to get everything exactly right, focus on the longer-term business goals when setting technical goals.

Predicting the Path

A hurricane doesn't move in a straight line or at a constant speed. As the storm progresses, the hurricane meteorologist issues updates with different projected paths, each one adjusted incrementally rather than radically from the previous one. An architecture's path is similar: the architect takes the architecture in

projected path, most meteorologists also publish how wrong they think it's likely to be. This deviation is expected and acceptable. Architects, especially agile architects, need to learn this—that it's okay to be a "little" wrong. Even if an architect got lucky and was right yesterday, she has no guarantee that she'll be right tomorrow. It also doesn't mean she was truly wrong yesterday. Of course, it isn't good if the architect is a lot wrong. If a hurricane meteorologist predicts landfall in southern Florida, and New York is hit instead,



ABOUT THE AUTHOR



ERIC RICHARDSON is the architect for editorial systems at Chemical Abstracts Service. His interests include software engineering, distributed systems, and applying agile principles to evolve architectures. Richardson has an MS in computer science from Franklin University. He's a member of IEEE and the ACM. Contact him at eric.richardson@cas.org.

what would happen? No one would listen to that meteorologist again.

As agile architects, we know the target isn't going to remain constant. Moreover, we know we're going to be wrong about its location. Nevertheless, an estimate of direction is still necessary because there's work to do, so direct the development teams to take the architecture closer to where it should end up.

Creating a Projected Path

The hardest part of the agile process is deciding how to get to where you think you want to go. The architect needs to create a plan so that the development team can take action immediately. The design should be specific where it matters and flexible where it doesn't. Junior development staff should have very precise direction on what to produce. Senior developers need less concrete guidance, but they still need to know about variables. If you're unsure whether you'll use SOAP, REST, RMI, Corba, or some other standard, your team leaders must understand this uncertainty to help them with subsequent migration when you finally do set architectural standards. Gather the data you can, analyze it, make a guess, and then tell people it's a guess.

Hurricanes follow the prevailing or steering winds. They go with the flow. As Ruth Netting notes, "When the steering winds are strong, it is easier to predict where a hurricane will go.

When the steering winds are weak, a storm seems to take on a mind of its own, following an erratic path that makes forecasting very difficult" (<http://kids.earth.nasa.gov/archive/hurricane/movement.html>). The target architecture is influenced by steering winds as well. When a particular technology or set of technologies really take hold and gain momentum and mind-share, it's easy to follow along (think Web 2.0). When a technology emerges and gets some adoption, it's more difficult to decide to use it (think CORBA).

Wind shear is the difference in speed and direction of winds at different altitudes. According to the Weather Underground website, "Wind shear is often the most critical factor controlling hurricane formation and destruction" (www.wunderground.com/education/shear.asp). Corporate culture, technical inertia, and legacy systems all influence an architectural effort's impact. Just as shearing winds can tear a hurricane apart, culture, inertia, and intractable legacy systems can destroy an architecture program. However, these influences aren't always negative—wind shear is what keeps most hurricanes from reaching Category 5. Technical wind shear can lead to investigations into alternate solutions or compromises to lessen the impact on legacy systems. At best, wind shear encourages the architect to factor in other priorities and viewpoints when making decisions.

One huge advantage the agile archi-

tect has over the hurricane meteorologist is the ability to shape the forces at play. Some of the steering winds affecting architectural direction are project managers and senior technical staff. Working with and influencing them is essential. If they're working at cross-purposes, they become destructive shearing winds. The agile architect must educate so that even the most intractable techie will at least understand, even if they don't agree with the architectural decision. Disagreement is a good thing. The agile architect must remember wind shear is an excellent moderating force; too much is what causes problems.

At CAS, many people have a seat at the table when we discuss architecture. Business leaders, technical managers, and development leads from both legacy and new systems all have input on architectural direction. As the agile architect at these meetings, I learn about business drivers and technical trade-offs made in legacy systems and I share the possibilities afforded by modern methodologies and technologies. Together, we're planning an acceptable path to sustainable architecture. By adopting the serenity and vigilance of a hurricane meteorologist—identifying the forces, some controllable, some not, that steer our architecture and continually assessing the impact—the agile architect can respond to changing conditions with an incremental, agile approach that keeps us able to respond to the next change. ☞

References

1. L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, Addison-Wesley Professional, 1998.
2. P. Abrahamsson, M. Ali Babar, and P. Kruchten, "Agility and Architecture: Can They Coexist?" *IEEE Software*, vol. 27, no. 2, 2010, pp. 16–22.
3. S. Tockey, *Return on Software: Maximizing the Return on Your Software Investment*, Addison-Wesley, 2008.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.